

De Manual a Escalable

Automatización RPA y Arquitectura Medallón para el Tablero del Macroentorno Ecuatoriano

Jesús Alejandro Rivas Reinoso

*Carrera de Computación, Universidad
Técnica Particular de Loja (UTPL)*

Abstract—Obtener datos es solo el primer paso; el verdadero desafío está en transformarlos, estructurarlos y garantizar que el proceso se repita sin depender de que una persona recuerde cómo hacerlo. Este artículo documenta el desarrollo y cierre de un pipeline de datos institucional construido en la DGTITD de la Universidad Técnica Particular de Loja: el Pipeline del Macroentorno Ecuatoriano, que integra nueve fuentes públicas (BCE, INEC, Supercías, MINEDUC) bajo la arquitectura Medallón. En la capa Bronze, un componente de RPA automatiza la extracción de las fuentes y las deposita empaquetadas en formato JSON dentro de una base Oracle. En la capa Silver, un script maestro en Python consume ese repositorio mediante oracledb, aplanar los JSON anidados con pandas (json_normalize), corrige la codificación de caracteres y carga los datos por lotes en una base relacional con modelo estrella (dimensiones dim_geografia y dim_tiempo compartidas). En la capa Gold, siete vistas SQL calculan los indicadores que responden a tres preguntas estratégicas y alimentan un dashboard interactivo en Power BI vía ODBC. Todo el flujo se orquesta con Prefect. El pipeline procesó más de 1,5 millones de registros del ranking empresarial y 212.000 del directorio de compañías, y su hallazgo de mayor valor —el cruce entre bachilleres de tercer año y empresas activas por provincia— revela brechas regionales entre la oferta de talento joven y la capacidad de absorción del tejido empresarial.

I. Introducción

Recopilar datos no es suficiente. El valor analítico de cualquier fuente depende de cuánto esfuerzo se invierte en limpiarla, estructurarla y hacerla coherente con otras fuentes. Majid et al. [3] estiman que la adquisición y preparación de datos consumen hasta el 80% del tiempo en proyectos analíticos reales, y Biswas et al. [6] confirman que estos pasos están presentes en prácticamente todos los flujos de trabajo, sin importar el dominio. A pesar de ello, en muchas organizaciones estos procesos siguen siendo manuales, no documentados y dependientes de personas específicas, lo que los vuelve frágiles ante cualquier cambio en las fuentes o en el equipo.

Ese fue exactamente el escenario que motivó este trabajo. La DGTITD de la UTPL mantenía un Tablero Macroentorno que integra indicadores económicos y laborales de nueve fuentes públicas ecuatorianas, pero su actualización era

manual y su pipeline no estaba documentado ni era reproducible: cuando cambiaba un archivo fuente, el conocimiento de qué hacer vivía en personas, no en código. El objetivo del prácticum en el que se enmarca este artículo fue construir el pipeline que hace ese proceso sostenible y reproducible, desde los archivos crudos que entrega el equipo de RPA hasta un dashboard que responde a tres preguntas analíticas concretas. Ese objetivo se cumplió: el pipeline hoy está implementado, orquestado y operando sobre datos reales. El sistema sigue la arquitectura Medallón en sus tres capas: Bronze (datos crudos consolidados por RPA en Oracle), Silver (modelo relacional en estrella sobre SQLite) y Gold (vistas de negocio que consume Power BI).

Este artículo revisa la literatura que sustentó las decisiones de diseño del pipeline y muestra cómo cada una se materializó, en torno a tres preguntas: (1) ¿Qué arquitecturas organizan mejor el flujo desde datos crudos hasta indicadores listos para visualizar? (2) ¿Cómo estructurar el modelo relacional de la capa Silver para garantizar calidad y mantenibilidad? y (3) ¿Qué enfoques documentados son aplicables a pipelines institucionales con recursos tecnológicos acotados? El sistema construido, que procesa las nueve fuentes de extremo a extremo, sirve como caso de contraste concreto a lo largo de la revisión.

II. MARCO CONCEPTUAL

A. Pipelines de Datos

Un pipeline de datos es una serie de pasos de procesamiento automatizados que mueven, transforman y entregan información desde su origen hasta un destino apto para análisis. Según Majid et al. [3], el propósito central es transformar datos crudos en una forma estructurada que sirva de entrada a tareas posteriores; estas actividades consumen hasta el 80% del tiempo de un proyecto de IA. Biswas et al. [6] encontraron que la adquisición, preparación y almacenamiento están presentes en prácticamente todos los flujos reales. La experiencia de este proyecto confirma esa proporción: el aplanado de JSON anidados, la corrección de codificación y la normalización de nombres de columna concentraron la mayor parte del esfuerzo, muy por encima de la extracción o la visualización.

B. Arquitecturas ETL y ELT

Históricamente, la ingeniería de datos operó bajo el paradigma Extraer–Transformar–Cargar (ETL): los datos se extraen, se transforman en un servidor intermedio y se cargan en la base destino. Su dependencia de una capa de transformación separada introduce cuellos de botella al manejar datos de alta velocidad o no estructurados [4]. La aparición de almacenes en la nube habilitó el paradigma ELT (Extraer–Cargar–Transformar), donde los datos crudos se cargan directamente y las transformaciones se ejecutan después. Armbrust et al. [4] demuestran que la arquitectura Lakehouse permite tanto la retención flexible de datos crudos como transformaciones de alto rendimiento sin servidor ETL dedicado. La solución implementada resultó un híbrido coherente con ambos paradigmas: la capa Bronze es netamente ELT (el RPA carga el JSON crudo en Oracle sin transformarlo, preservando el dato como landing zone al estilo Lakehouse [4]), mientras que el tránsito Bronze → Silver → Gold es ETL clásico, porque Python transforma antes de cargar en una base SQLite ligera. Esta elección es coherente con lo que Majid et al. [3] describen como pipelines orientados

a entornos con infraestructura limitada, donde la simplicidad operativa vale más que la escalabilidad horizontal.

C. Orquestación de Datos

La orquestación coordina automáticamente múltiples tareas interdependientes dentro de un pipeline. Apache Airflow implementa el paradigma Workflows as Code mediante grafos acíclicos dirigidos (DAGs) en Python, y lo usan organizaciones como Google, Adobe y PayPal [5]. Mir et al. [5] documentan, sin embargo, que la configuración y el mantenimiento de Airflow presentan una curva de aprendizaje considerable, lo que ha impulsado alternativas más ligeras como Prefect. Esa tensión se resolvió aquí a favor de Prefect: Airflow habría exigido desplegar un scheduler, una base de metadatos y un servidor web para orquestar un flujo de seis tareas, un costo desproporcionado para una unidad con recursos acotados. Los decoradores `@task` y `@flow` convirtieron las funciones Python existentes en un flujo orquestado sin reescribir la lógica ni añadir infraestructura; el beneficio central no fue programar horarios, sino eliminar los fallos silenciosos, ya que cada tarea reporta su estado de forma individual.

Rana et al. [1] ofrecen una visión de conjunto del estado de la ingeniería de datos, subrayando que la disciplina ha madurado más en sus herramientas que en sus prácticas de documentación y reproducibilidad, una brecha que este proyecto buscó cerrar al formalizar en código un proceso que antes residía en la memoria de sus operadores. Majid et al. [3] categorizan las soluciones en cuatro grupos: pipelines ETL/ELT, herramientas de ingesta y transformación, plataformas de orquestación y pipelines de aprendizaje automático; el sistema construido cubre los tres primeros y deja el cuarto como trabajo futuro. Heck [2] mapea las actividades de ingeniería de datos a lo largo del ciclo de vida de sistemas de IA, identificando la preparación y transformación como las más estudiadas, con escasez relativa en gobernanza y observabilidad.

Armbrust et al. [4] introducen la arquitectura Lakehouse, que combina la flexibilidad de los data lakes con el rendimiento de los almacenes tradicionales. Mir et al. [5] realizan un estudio empírico sobre los desafíos de implementar Workflows as Code en Airflow, analizando más de 9.000 publicaciones en Stack Overflow. Biswas et al. [6] confirman empíricamente la universalidad de los patrones de adquisición y preparación de datos. Ghosh et al. [12] sitúan a la RPA como habilitador de la recolección automatizada de datos, y Bala Krishna y Kumar [13] fundamentan el modelado en estrella como técnica de diseño para almacenes analíticos.

IV. CASOS DE ESTUDIO Y APLICACIONES

A. Entornos de Smart Campus Universitario

Hernando-Cánovas et al. [7] desarrollaron un sistema predictivo para estimar la ocupación física en una biblioteca universitaria usando datos anónimos de conexiones WiFi. El pipeline extrae continuamente telemetría de puntos de acceso, la procesa y alimenta modelos de aprendizaje automático para producir estimaciones sin cámaras ni datos personales. Los

autores concluyen que la solución es robusta, rentable y preserva la privacidad [7], ilustrando cómo un pipeline bien estructurado transforma telemetría de bajo nivel en inteligencia operativa. El paralelismo con el presente trabajo es directo: en ambos casos el valor no surge del dato individual sino de la capa que lo agrega y lo contextualiza dentro del campus.

B. Industria Financiera

Ahonen [9] desarrolló una arquitectura para integrar datos de sistemas bancarios heredados hacia plataformas cloud, usando Apache Spark Structured Streaming con Azure Event Hubs para capturar eventos continuos y mantener un perfil de cliente unificado, con latencias de 1,2 a 1,4 segundos [9]. El contraste con este proyecto es instructivo: el macroentorno ecuatoriano se publica con periodicidad mensual, trimestral o anual, de modo que la latencia de segundos no aporta valor y una arquitectura de streaming habría sido sobreingeniería. La decisión correcta fue la opuesta: procesamiento por lotes sobre una base ligera.

V. HERRAMIENTAS Y TECNOLOGÍAS

La Sección V describe la arquitectura implementada capa por capa. La Tabla I resume las herramientas efectivamente empleadas, alineadas con la taxonomía de Majid et al. [3].

TABLE I. TABLA I

TABLE II. STACK TECNOLÓGICO IMPLEMENTADO EN EL PIPELINE

Herramienta	Función en el pipeline	Rol efectivo en la implementación
RPA (Capa Bronze)	Extracción automatizada	Descarga las 9 fuentes públicas y las deposita como JSON en TAB_CONSOLIDADO (Oracle), eliminando la descarga manual [12]
Oracle oracledb	+ Landing zone y conectividad	Oracle almacena el JSON crudo sin esquema previo; oracledb lo extrae por lotes desde Python [4]
Python / pandas	Lógica y limpieza	json_normalize aplanar los JSON anidados; corrección de codificación, tipado y normalización [3]
SQLite	Almacén Silver / Gold	Base relacional local liviana, coherente con entornos de infraestructura acotada [3]
SQL (Modelo estrella)	Modelo relacional	dim_geografia y dim_tiempo como dimensiones compartidas; hechos con claves foráneas [13]
Prefect	Orquestación	Decoradores <code>@task</code> y <code>@flow</code> ; trazabilidad por tarea sin la curva de Airflow [5]
SQL (Vistas Gold)	Indicadores de negocio	Siete vistas analíticas ejecutadas desde Python que responden las 3 preguntas del dashboard
Power (ODBC)	BI Visualización	Consume las vistas Gold y las convierte en indicadores interactivos de tres páginas [1]

VI. RECOMENDACIONES Y TRABAJO FUTURO

IA predictiva sobre la capa Gold: la capa Gold es, por construcción, un conjunto de series históricas limpias, tipadas y consistentes, exactamente el insumo que Majid et al. [3] identifican como la barrera habitual para los pipelines de aprendizaje automático y que aquí ya está resuelta. La

extensión natural es entrenar modelos predictivos sobre estas vistas: proyectar el ratio de `gold_bachilleres_vs_empresas` para anticipar en qué provincias la escasez de tejido empresarial no absorberá a los futuros graduados, y usar `gold_pib_tendencia` y `gold_empleo_tendencia` para modelos de series temporales. Esto alinearía el pipeline con la tendencia de datos listos para IA que documenta Heck [2].

Automatización proactiva mediante alertas: el dashboard actual es pasivo, informa a quien lo consulta. Dado que el flujo ya está orquestado con Prefect, incorporar una tarea de vigilancia posterior a las vistas Gold es de bajo costo: evaluaría umbrales críticos —una caída del PIB, un repunte del riesgo país, un descenso del empleo adecuado— y enviaría alertas automáticas por correo a los directivos. Este enfoque materializa lo que Ehrlinger y Rusz [10] describen como observabilidad continua dentro del stack del pipeline, extendiendo la vigilancia desde la calidad técnica del dato hacia el valor de negocio del indicador.

Escalabilidad en la nube: SQLite fue la elección correcta para la construcción (cero costo, cero administración, suficiente para el volumen actual), pero su límite es la concurrencia y el acceso distribuido. Migrar la capa Silver/Gold a un servicio gestionado (AWS RDS, Azure SQL Database o Google BigQuery) democratizaría el acceso al lago de datos para que Planificación, Vinculación y las facultades consulten los mismos indicadores en tiempo real. La arquitectura Medallón facilita la transición: al estar la lógica desacoplada del destino, migrar implica sustituir la función de carga, no reescribir el pipeline, acercando la solución al modelo Lakehouse de Armbrust et al. [4] y el borde documentado por Hamid et al. [11].

VII. CONCLUSIONES

La transformación fue el núcleo del pipeline: la literatura anticipaba que la preparación y limpieza de datos es la etapa más costosa y crítica de cualquier proyecto analítico [3], [6], y el desarrollo lo confirmó. El aplanado de JSON anidados, la corrección de codificación (`mojibake cp1252 → utf-8`) y la normalización de esquemas concentraron la mayor parte del esfuerzo. Además, la automatización de la ingesta con RPA [12] eliminó el punto más frágil del flujo anterior —la descarga manual— y fijó una frontera limpia entre el equipo de RPA y el de transformación en la tabla `TAB_CONSOLIDADO`.

La arquitectura define lo que es mantenible: la separación en capas Bronze, Silver y Gold, respaldada por Armbrust et al. [4] y Majid et al. [3], sumada a un modelo estrella con dimensiones compartidas `dim_geografia` y `dim_tiempo` [13], no es una decisión estética sino funcional. Permitió que ambos equipos trabajaran de forma independiente y que los ajustes en las reglas de limpieza no obligaran nunca a reprocesar la ingesta. La orquestación ligera con Prefect [5] aportó lo esencial —trazabilidad por tarea y ausencia de fallos silenciosos— sin exigir infraestructura adicional.

El resultado más valioso no requirió técnica sofisticada: los trabajos de Hernando-Cánovas et al. [7] y Ahonen [9] muestran que el factor diferenciador no es la sofisticación del algoritmo sino la solidez del pipeline que lo alimenta. El cruce entre bachilleres y tejido empresarial por provincia —

que revela brechas regionales de absorción de talento, con Bolívar (7,14), Esmeraldas (6,83) y Los Ríos (4,86) a la cabeza y Pichincha (0,92) y Guayas (1,12) en equilibrio— no necesitó aprendizaje automático: solo dos fuentes limpias y una unión bien planteada. Esa lección es directamente aplicable al contexto institucional en el que se desarrolla este trabajo.

Reproducibilidad como objetivo de diseño: documentar las decisiones de transformación y formalizar el pipeline en código convirtió un proceso frágil, dependiente de la memoria de sus operadores, en uno sostenible y ejecutable por cualquier persona con acceso al repositorio. Ese es el punto de partida —y no la meta— de la analítica institucional: la capa Gold que hoy alimenta un tablero de tres páginas es la misma que mañana entrenará modelos predictivos y disparará alertas automáticas.

REFERENCIAS

- [1] D. Rana, A. Bhatt, R. B. Dubey y S. K. Bhatt, “Data Engineering: An Overview from a Future Perspective,” en *Proc. 2023 IEEE Int. Conf. on Contemporary Computing and Communications (InC4)*, IEEE, 2023. DOI: 10.1109/InC4.2023.10398128
- [2] P. Heck, “What About the Data? A Mapping Study on Data Engineering for AI Systems,” en *Proc. CAIN 2024*, ACM, Lisboa, abr. 2024. DOI: 10.1145/3644815.3644954
- [3] A. Majid, S. Laue, K. Böhm y N. Piatkowski, “A Survey of Pipeline Tools for Data Engineering,” *arXiv preprint arXiv:2406.08335*, 2024.
- [4] M. Armbrust et al., “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics,” en *Proc. 11th CIDR 2021*, 2021.
- [5] H. Mir, D. Rajbhadur y Y. Tian, “An Empirical Study of Developers’ Challenges in Implementing Workflows as Code: Apache Airflow,” *J. Systems and Software*, vol. 217, 2024. DOI: 10.1016/j.jss.2024.112029
- [6] S. Biswas, M. Wardat y H. Rajan, “The Art and Practice of Data Science Pipelines,” en *Proc. ICSE 2022*, pp. 2091–2103. DOI: 10.1145/3510003.3510057
- [7] L. Hernando-Cánovas et al., “A Machine Learning Approach for Estimating Person Counts Using Anonymous WiFi Data in a University Library,” *Sensors*, vol. 25, n.º 22, p. 7065, 2025. DOI: 10.3390/s25227065
- [8] A. Alzahrani, B. Alharbi y K. Al-Tawil, “Apache Spark in Riot Games: A Case Study on Data Processing,” *IJACSA*, vol. 14, n.º 7, 2023. DOI: 10.14569/IJACSA.2023.0140704
- [9] J. Ahonen, “Real-Time Customer Data Platform using Apache Spark Structured Streaming and Databricks,” Tesis de Máster, Tampere University, 2025. *Tesis de Máster, Tampere University*, 2025.
- [10] L. Ehrlinger y W. Rusz, “A Survey of Data Quality Measurement and Monitoring Tools,” *Frontiers in Big Data*, vol. 5, 2022
- [11] A. Hamid et al., “Enhancing IoT Efficiency by Minimizing Latency with Edge Computing,” *SN Computer Science*, Springer, 2025. DOI: 10.1007/s42979-025-0419

- [12] S. Ghosh, R. Shetty et al., "Robotics Process Automation Technologies for Data Collection and Analysis," en Proc. 2024 IEEE Int. Conf., IEEE, 2024. DOI: 10.1109/ICKECS.2024.10725006
- [13] R. Bala Krishna y S. Kumar, "Dimensional Modeling Using Star Schema for Data Warehouse Creation," Oriental J. of Computer Science and Technology, vol. 10, n.º 4, 2017.*SN Computer Science*, Springer, 2025. DOI: 10.1007/s42979-025-04192-x